

Chapter 3: ETL Processes

Extract, Transform, Load — Starting with Extract

Learning Objectives

By the end of this chapter, you should be able to:

1. Explain why extraction is not simply copying data, but a deliberate process involving connection, format, and scope decisions.
2. Distinguish how extraction differs across genuinely different source types (a relational database versus a spreadsheet).
3. Implement working Python extraction routines, using pandas and SQLAlchemy, for two different source systems in a university admissions scenario.

A Guided Question to Begin

Picture an IT staff member building the very first stage of a university admissions data warehouse's ETL pipeline: Extract. The admissions office has two real sources of applicant data. An online application portal, a proper relational database, where applicants fill out a structured form themselves. And a walk-in intake process, where staff manually type paper applications into a simple spreadsheet as applicants arrive in person.

Confident that “extract” simply means “connect to each source and copy everything,” the staff member writes one quick script pulling every column from the portal's database, and another pulling every column from the walk-in spreadsheet, dumping both, unchanged, into a single staging file. The result is a mess almost immediately. The portal's data includes internal fields, staff routing notes, admin flags, nobody actually needs in the warehouse. The walk-in spreadsheet's “Course Applying For” column is free-typed, riddled with abbreviations and typos (“BSIT,” “BS IT,” “bsit,” “Bachelor of IT”), where the portal's identical field is a clean, consistent dropdown value.

Before scrolling further: did copying every column from both sources actually count as extraction? Or was something essential about the extract step itself skipped entirely?

Introduction

Chapter 2 established the full architecture a data warehouse actually needs, source systems, staging, the warehouse, marts, held together by metadata. This chapter takes apart the actual mechanism that moves data through the first two of those layers: ETL, Extract, Transform, Load, the process that takes data as it exists in real, messy source systems and turns it into something the staging area can actually trust.

It's tempting to treat ETL's first letter as the simplest one: extract, meaning, presumably, just copy the data out of wherever it currently lives. This chapter corrects that assumption directly, since extraction, done properly, is already making real, consequential decisions, well before transformation ever begins.

This chapter uses a university admissions office consolidating two genuinely different sources, an online portal and a walk-in intake spreadsheet, as its running example, and it's this course's first genuinely hands-on chapter: you'll write real Python code connecting to a real MySQL database.

3.1 ETL: Three Stages, Not One Copy Operation

It's worth naming all three stages of ETL clearly before focusing on the first. Extraction is the process of collecting raw data from different sources, databases, spreadsheets, APIs, or other systems, often arriving in genuinely different formats and structures (Airbyte, 2025).

Transformation, covered in a later chapter, is where that extracted data actually gets cleaned, standardized, and reconciled. Load is where the transformed result actually gets written into the warehouse.

This chapter stays with Extract specifically, since it's the stage most often treated as trivial, and the stage where a naive approach quietly sets up problems the later stages then have to work around.

3.2 Extract Is Not “Just Copy”: The Admissions Office's Two Sources

Return to the IT staff member's admissions pipeline, and to what “copy everything from both sources” actually produced.

Two genuinely different sources, requiring two genuinely different extraction approaches, got treated identically. The online portal is a structured relational database; extracting from it means writing a query, connecting through a database driver, and pulling specific tables and columns. The walk-in intake spreadsheet is an entirely different kind of source, hand-typed, inconsistent, requiring a completely different extraction method, reading a spreadsheet file directly, since its columns don't even match the portal's column names, let alone its data quality.

3.3 Extraction as Receiving a Letter

Here's a way to hold what actually distinguishes a genuine extraction step from a simple copy: think of ETL the way you'd think of handling an important letter written in a language you don't fully read yourself.

Extract is receiving the letter, physically getting it into your hands from wherever it currently is, a mailbox, a courier, a scanned attachment. That's real work, and it looks different depending on how the letter actually arrives. Transform is translating it, and, just as importantly, verifying the translation actually says what you think it says. Load is filing the now-translated, verified letter somewhere it can actually be found again later. Skip the translation step, and you've filed a letter nobody can trust or read.

But notice: even the “simple” first step, receiving the letter, already requires knowing whether it arrived by mail, by courier, or as a scanned file, since each one is retrieved differently. Extraction is exactly this first step, and it's already doing real, source-specific work before anything gets transformed at all.

3.4 What Extraction Actually Requires

It's worth naming precisely what a genuine extraction step has to decide, since “connect and copy” skips over all of it.

Connection: how do you actually reach this specific source? A relational database needs a connection string, credentials, and a query language. A spreadsheet needs a file path and a library that knows how to read its specific format. These are not interchangeable, and code written for one will not work against the other.

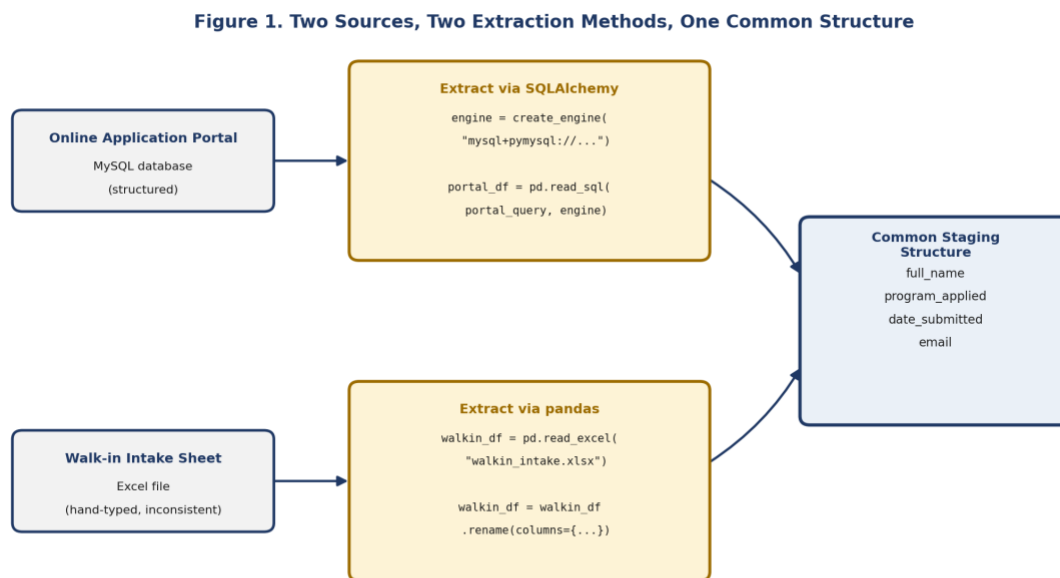
Format: what shape does this source's data actually arrive in? Source data can be structured, like a SQL database, or considerably less structured, like a hand-typed spreadsheet, and an extraction routine needs to account for that difference before a single value gets compared between sources (Portable.io, 2026).

Scope: which specific fields does the warehouse actually need? The portal's internal staff-routing notes and the walk-in sheet's "processed by" initials column are real data, but they're not data the admissions warehouse's actual analytical purpose requires. Extracting everything indiscriminately doesn't just waste space; it drags irrelevant, sometimes sensitive, internal information into a staging area that was never meant to hold it.

Pause and reflect: Think of two real data sources you're familiar with, even informally, that hold overlapping information (a spreadsheet and an app, for instance). What is one field each source has that the other doesn't, and would a warehouse combining them actually need both?

3.5 Building the Extraction Step in Python

Figure 1 traces the two source-specific extraction routines the admissions pipeline actually needs, side by side, each connecting to its source the way that source specifically requires.



Each source needs its own connection method and its own scope decisions — but both extraction routines are written to produce the identical output structure.

Figure 1. Two different extraction methods, one for each source's actual format, both producing the identical output structure the staging area expects.

For the online portal, a structured database, extraction means connecting via SQLAlchemy and pulling only the specific columns the warehouse actually needs (whyamit404, 2025):

```
import pandas as pd
from sqlalchemy import create_engine

# Connect to the online application portal's MySQL database
portal_engine = create_engine(
    "mysql+pymysql://etl_user:password@localhost/admissions_portal"
)

# Extract only the fields the warehouse actually needs
portal_query = """
    SELECT applicant_id, full_name, program_applied,
           date_submitted, email
    FROM applications
"""
portal_df = pd.read_sql(portal_query, portal_engine)
```

For the walk-in intake spreadsheet, extraction means a genuinely different approach, reading a file directly rather than querying a database, and immediately renaming its inconsistent columns to a common structure (PyQuestHub, 2025):

```
import pandas as pd

# Extract from the walk-in intake spreadsheet
walkin_df = pd.read_excel("walkin_intake_2026.xlsx")

# Rename inconsistent columns to match the portal's structure
walkin_df = walkin_df.rename(columns={
    "Applicant Name": "full_name",
    "Course Applying For": "program_applied",
    "Date Received": "date_submitted",
    "Email Address": "email"
})

# Drop columns the warehouse doesn't need (e.g., staff routing notes)
walkin_df = walkin_df[["full_name", "program_applied",
                      "date_submitted", "email"]]
```

Notice that neither routine simply “copies everything.” Each one is already making a deliberate choice about which columns matter, and each one is already written specifically for its own

source's actual format. This is extraction done correctly, and it's the necessary foundation the Transform step, covered in a later chapter, will build on directly.

3.6 Where Transform and Load Pick Up From Here

This chapter deliberately stopped at Extract, since giving this first stage its actual due is exactly what most naive ETL attempts skip past. The “Program Applied For” inconsistency, “BSIT” against “BS IT” against “bsit,” is real, and it's genuinely still a problem in both DataFrames above. Fixing it isn't extraction's job. It belongs to Transform, the stage that actually reconciles, standardizes, and validates data, now that extraction has correctly pulled the right, source-appropriate data out of both systems in the first place.

Common Misunderstandings to Watch For

Each item below reflects a version of this chapter's central lesson, applied to a slightly different situation. Reviewing them now is good practice for this week's assessment.

1. Treating extraction as a single, universal copy operation, rather than recognizing each source requires its own connection method and format handling.
2. Extracting every available column from a source, rather than deliberately deciding which fields the warehouse's actual purpose requires.
3. Writing one extraction routine and assuming it will work against any source, rather than recognizing a database and a spreadsheet require genuinely different code.
4. Assuming an extraction problem (like inconsistent values) belongs to the Extract stage, rather than recognizing that reconciliation is Transform's job.

Key Terms Introduced This Chapter

Extract: The ETL stage that retrieves data from source systems, requiring source-specific decisions about connection method, data format, and field scope.

Connection method: The specific way code reaches a given data source: a database connection string and query for a relational database, a file path and reader function for a spreadsheet.

Scope (in extraction): The deliberate decision about which fields a warehouse actually needs from a source, as distinct from extracting every available column.

Summary

Go back to that first extraction attempt one more time: two sources, one script treating them identically, and a staging file bloated with irrelevant admin columns and free-typed inconsistencies that were never going to be caught by “connect and copy.” You now have language for why that approach fell short before transformation even had a chance to help. Extraction was never simply retrieval. It's the stage where you decide how to actually reach each specific source, what shape its data actually takes, and which of its fields the warehouse genuinely needs.

You can also say, now, why receiving a letter is real work, even before anyone translates a single word of it. Knowing whether a letter arrived by mail, courier, or scan changes how you actually get it into your hands, the same way knowing whether a source is a database or a spreadsheet changes how you actually extract from it.

Carry this distinction into every ETL pipeline you build for the rest of this course. Extraction done well is already doing real, deliberate work. Extraction done as a blind copy operation simply moves a source's mess one step closer to a warehouse that was supposed to be trustworthy.

Performance Task

Title: Extracting from Two Real Sources

For this task, you will write working Python extraction code for two genuinely different data sources.

Your submission should include the following parts:

5. Choose one real or realistic scenario (your own OLIMFBDS case study, the admissions office, or another organization) with at least two genuinely different source systems, for example, a database and a spreadsheet, or two databases with different schemas.
6. For each source, identify its actual connection method, its data format, and the specific subset of fields your warehouse would genuinely need from it, not simply “all of them.”
7. Write two short Python extraction routines (using pandas, and SQLAlchemy where applicable) that pull the identified subset of fields from each source into a common staging structure.

8. In 150 to 200 words, explain one specific way your two sources' data differs in shape or format, and how your extraction code accounts for that difference.

Submit your working code and written explanation through the LMS by the stated deadline.

Note: this task is evaluated on whether your two extraction routines are genuinely written for their specific sources' actual formats, not on how many sources or fields you include. Two clearly different sources, correctly and deliberately extracted, is exactly what this task is asking for.

References

Airbyte. (2025, August 10). ETL (Extract, Transform, Load) process: Steps & benefits.

<https://airbyte.com/data-engineering-resources/etl-process>

Portable.io. (2026). ETL process & best practices for data warehouse.

<https://portable.io/learn/etl-process-data-warehouse>

PyQuestHub. (2025, March 3). ETL process using pandas and SQLAlchemy: A comprehensive guide. <https://pyquesthub.com/streamlining-data-management-building-an-etl-process-using-pandas-and-sqlalchemy>

whyamit404. (2025, April 12). Pandas ETL: A practical guide for beginners. Medium.

<https://medium.com/@whyamit404/pandas-etl-a-practical-guide-for-beginners-f23fc19eca3d>